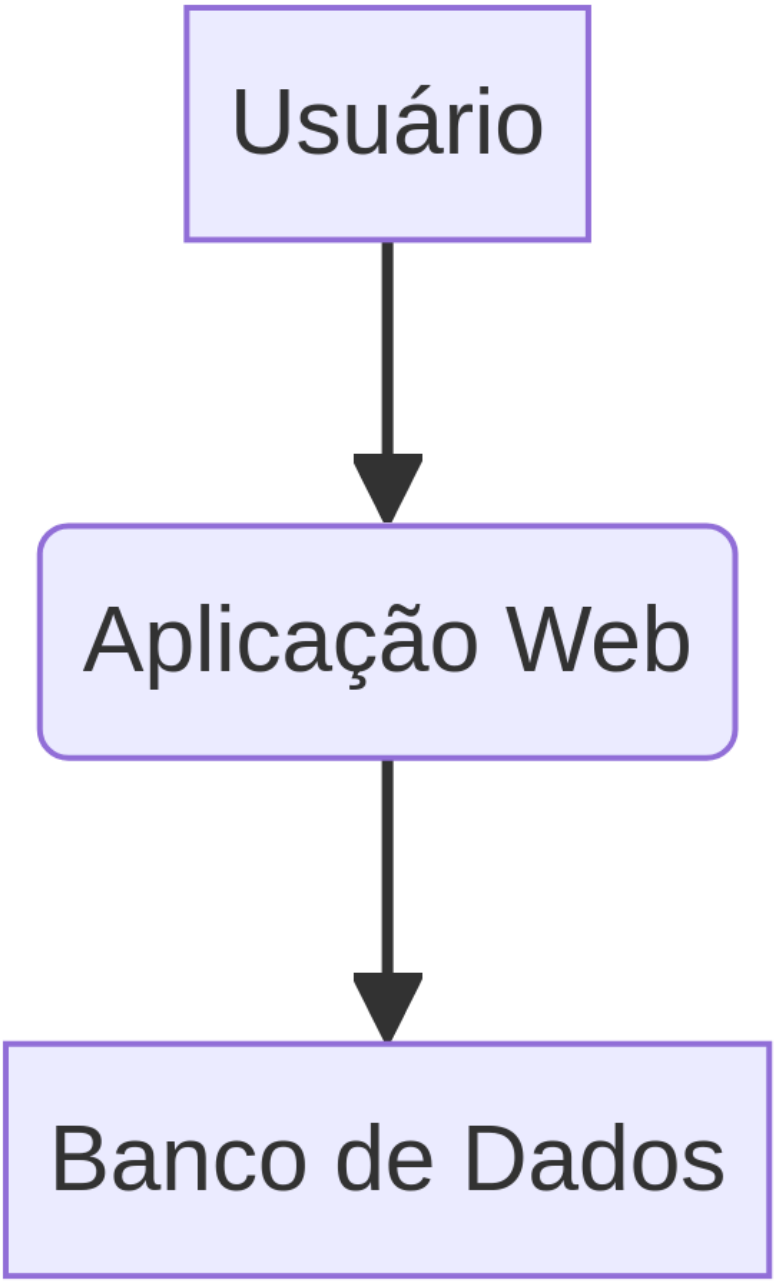
Arquiteturas de Software para Sistema de Help Desk

Este documento apresenta três exemplos de arquiteturas de software para um sistema de Help Desk, com níveis crescentes de complexidade de infraestrutura.

1. Arquitetura Básica

Esta arquitetura representa uma configuração simples, ideal para protótipos ou sistemas com baixa demanda. Geralmente, todos os componentes são executados em um único servidor ou em um ambiente de nuvem com recursos mínimos.

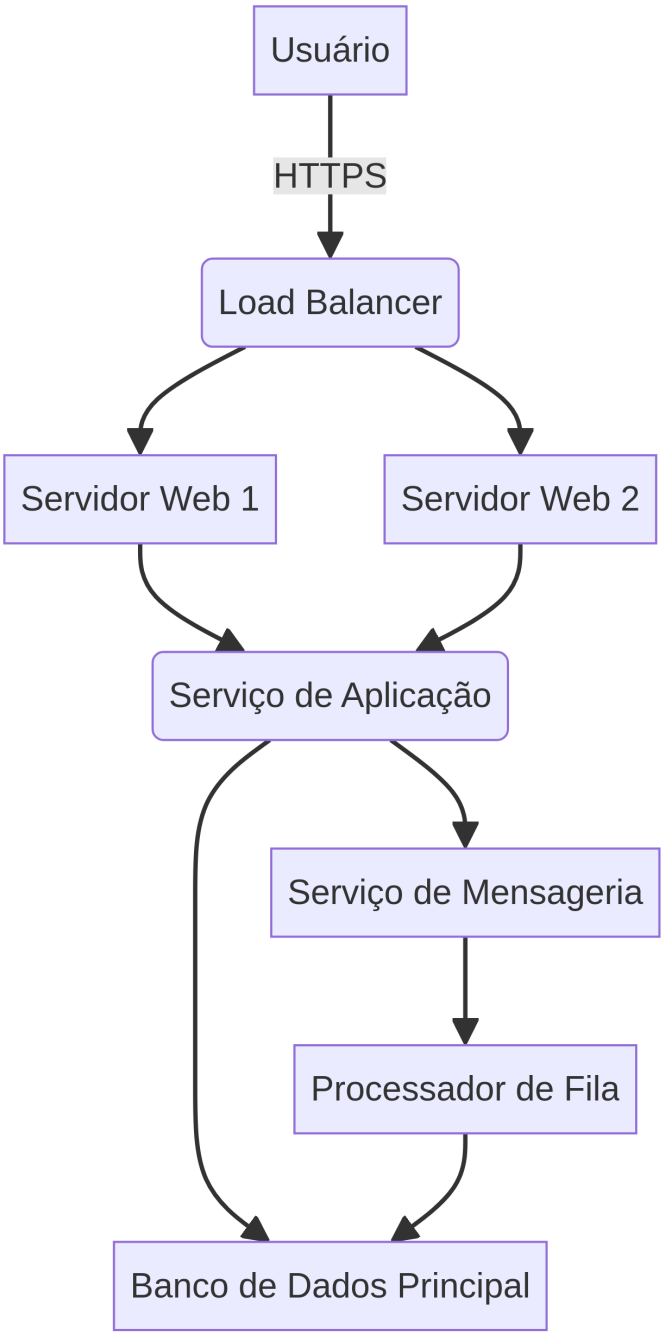


Componentes:

- **Usuário:** Interage diretamente com a aplicação.
- **Aplicação Web:** Contém a lógica de negócio e a interface do usuário.
- **Banco de Dados:** Armazena todos os dados da aplicação.

2. Arquitetura Intermediária

A arquitetura intermediária introduz conceitos de escalabilidade e resiliência, distribuindo a carga entre múltiplos servidores e adicionando serviços especializados para melhorar o desempenho e a confiabilidade. É adequada para sistemas com demanda moderada e que exigem maior disponibilidade.



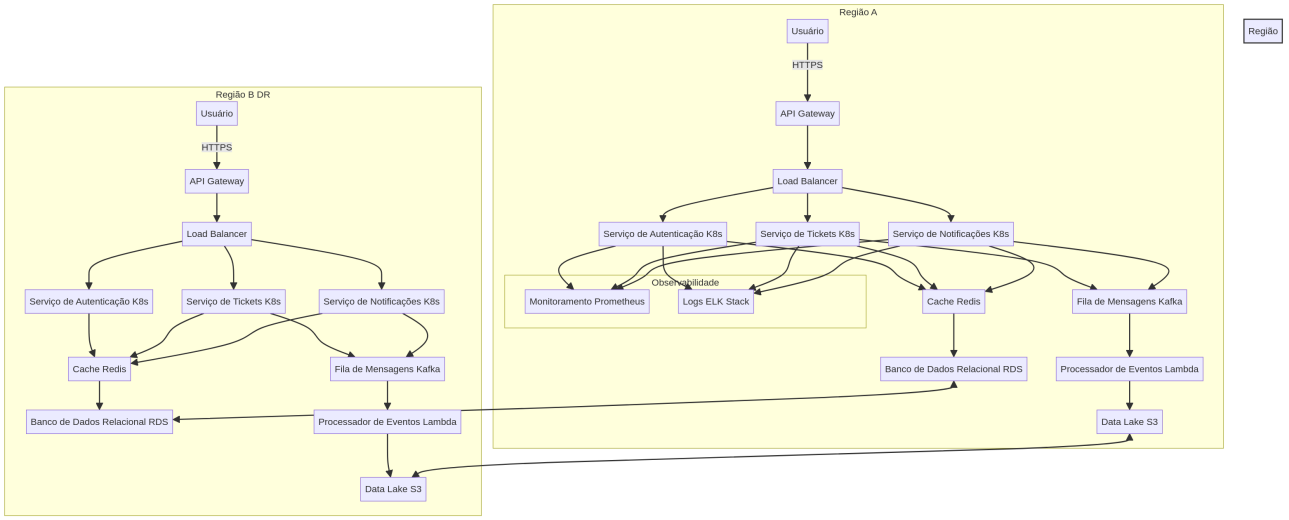
Componentes:

- **Usuário:** Acessa a aplicação via HTTPS.

- **Load Balancer:** Distribui as requisições entre os servidores web.
- **Servidor Web 1 e 2:** Instâncias da aplicação web para processar requisições.
- **Serviço de Aplicação:** Lógica de negócio principal, desacoplada dos servidores web.
- **Banco de Dados Principal:** Armazena os dados transacionais.
- **Serviço de Mensageria:** Gerencia a comunicação assíncrona entre componentes (ex: RabbitMQ, Kafka).
- **Processador de Fila:** Processa tarefas em segundo plano (ex: envio de e-mails, geração de relatórios).

3. Arquitetura Avançada

A arquitetura avançada é projetada para alta disponibilidade, escalabilidade massiva e recuperação de desastres, utilizando microsserviços, orquestração de contêineres (Kubernetes), serviços de nuvem gerenciados e observabilidade robusta. É ideal para sistemas críticos com grande volume de usuários e dados, e que operam em ambientes distribuídos globalmente.



Componentes:

- **Região A (Principal):**
  - **Usuário:** Acessa a aplicação via HTTPS.
  - **API Gateway:** Ponto de entrada unificado para os microsserviços.
  - **Load Balancer:** Distribui a carga entre os clusters Kubernetes.

    - **Serviço de Autenticação (K8s):** Gerencia a autenticação e autorização de usuários.
    - **Serviço de Tickets (K8s):** Lógica de negócio para gerenciamento de tickets.
    - **Serviço de Notificações (K8s):** Envia notificações aos usuários.
    - **Cache (Redis):** Armazena dados frequentemente acessados para reduzir a latência.
    - **Banco de Dados Relacional (RDS):** Banco de dados gerenciado para dados transacionais.
    - **Fila de Mensagens (Kafka):** Plataforma de streaming de eventos para comunicação assíncrona.
    - **Processador de Eventos (Lambda):** Funções serverless para processamento de eventos.
    - **Data Lake (S3):** Armazenamento de dados brutos para análise.
    - **Observabilidade:**
        - **Monitoramento (Prometheus):** Coleta métricas do sistema.
        - **Logs (ELK Stack):** Gerencia e analisa logs.
- **Região B (DR - Recuperação de Desastres):** Uma réplica da Região A, garantindo a continuidade do serviço em caso de falha da região principal. Os componentes são espelhados para garantir a resiliência.

Conectividade entre Regiões:

- **Banco de Dados Relacional (RDS):** Sincronização de dados entre as regiões.
- **Data Lake (S3):** Sincronização de dados entre os buckets S3 das regiões.